This document consolidates the **Fraud Use Cases** for the Fraud Intelligence Demo into a single, canonical reference. It is an expansion of `PRD/PRD.md` .

PRD (Expansion): Fraud Use Cases

Purpose

Provide a single place that describes each fraud use case in a **demo-ready** way:

- **Story** (what the investigator believes is happening)
- **Graph pattern** (topology + required collections/fields)
- **Signals** (what makes it suspicious)
- **Example AQL** (starter queries)
- **Demo steps** (what to click/show)
- **Test hooks** (what we assert in automated tests)

Naming & schema conventions

All names here follow `PRD/PRD.md` canonical naming:

- **Vertex collections (Classes)**: PascalCase (e.g., `Person` , `BankAccount`)
- **Edge collections (ObjectProperties)**: camelCase (e.g., `transferredTo` , `resolvedTo`)
- **Fields (DatatypeProperties)**: camelCase (e.g., `circleRateValue` , `riskScore`)

Use Case 1: Circular Trading ("Round Trip" Transfers)

Story

A group of accounts repeatedly move funds in a loop to inflate turnover and launder funds ("layering" behavior). In the real-estate version of this story, the loop may coincide with rapid property transactions and/or suspiciously consistent transfer amounts.

Graph pattern

- **Vertices**: `BankAccount` (optionally `Person`, `Organization`)
- **Edges**: `transferredTo`
- **Key fields** (on `transferredTo`):
  - `amount`, `timestamp`, `txnType`
  - optional tag for demo datasets: `scenario == "cycle"`

Signals

- Existence of a **directed cycle** of transfers.
- Tight timing window, repeated amounts, repeated counterparty set.

Example AQL (starter)

```
WITH BankAccount, transferredTo
LET start = FIRST(
  FOR e IN transferredTo
    FILTER e.scenario == "cycle"
    RETURN e._from
)
FILTER start != null
FOR v, e, p IN 1..6 OUTBOUND start transferredTo
  FILTER e.scenario == "cycle"
  FILTER v._id == start
  LIMIT 1
  RETURN p
```

Demo steps (Visualizer)

- Open **DataGraph** or **KnowledgeGraph**.
- Start from a `BankAccount` involved in the loop.
- Run “Find 2-hop neighbors (default)” and/or a 1-hop expansion to reveal the ring.
- Explain: “A closed walk exists; this is a classic circular-trading signature.”

Test hooks

- Integration test verifies at least one cycle exists (see `tests/test_fraud_patterns_integration.py`).

Use Case 2: Money Mule Ring (“Smurfing” Hub-and-Spoke)

Story

Many low-activity accounts (“mules”) forward funds to a single aggregator account quickly (e.g., within 24 hours). Mules often share a device/IP footprint.

Graph pattern

- **Vertices**: `BankAccount` , `DigitalLocation` (and optionally `Person`)
- **Edges**: `transferredTo` , `accessedFrom` (and optionally `hasDigitalLocation`)
- **Key fields**:
  - `transferredTo.scenario == "mule"` (demo tag)
  - `accessedFrom.accessType` , `accessedFrom.accessTimestamp`

Signals

- One `BankAccount` receives transfers from many distinct sources in a short window.
- Mule sources share a single `DigitalLocation` (shared IP/device).

Example AQL (starter)

```
WITH BankAccount, transferredTo
FOR e IN transferredTo
  FILTER e.scenario == "mule"
  COLLECT hub = e._to WITH COUNT INTO n
  SORT n DESC
  LIMIT 5
  RETURN { hub, n }
```

```
WITH BankAccount, DigitalLocation, transferredTo, accessedFrom
LET muleFrom = UNIQUE(
  FOR e IN transferredTo
    FILTER e.scenario == "mule"
    RETURN e._from
)
LET devices = UNIQUE(
  FOR a IN accessedFrom
    FILTER a._from IN muleFrom
    RETURN a._to
)
RETURN { muleSources: LENGTH(muleFrom), distinctDevices: LENGTH(devices) }
```

Demo steps (Visualizer)

- Start at the **hub** `BankAccount` (highest inbound mule count).
- Expand inbound `transferredTo` and show “many-to-one” structure.
- Expand mule accounts to show the shared `DigitalLocation` .

Test hooks

- Integration test expects a hub with ≥ 50 inbound mule transfers and exactly one shared digital location (see `tests/test_fraud_patterns_integration.py`).

Use Case 3: Undervalued Property Transactions (Circle Rate Evasion)

Story

Property sale transactions are recorded at or below the government minimum (“circle rate”) to evade taxes and launder value off-ledger (e.g., “cash component”).

Graph pattern

- **Vertices**: `RealProperty` , `RealEstateTransaction` , optionally `Document`
- **Edges**: `registeredSale` , optionally `mentionedIn`
- **Key fields**:

- `RealProperty.circleRateValue` , `RealProperty.marketValue`
- `RealEstateTransaction.transactionValue` , `RealEstateTransaction.paymentMethod`

Signals

- `transactionValue <= circleRateValue`
- `paymentMethod` indicates non-standard mix ("Mixed") for suspicious cases.
- Supporting unstructured evidence (if present) mentions "cash component".

Example AQL (starter)

```
WITH RealProperty, RealEstateTransaction, registeredSale
FOR e IN registeredSale
  LET p = DOCUMENT(e._from)
  LET tx = DOCUMENT(e._to)
  FILTER p != null && tx != null
  FILTER tx.transactionValue <= p.circleRateValue
  LIMIT 20
  RETURN {
    property: p._id,
    circleRateValue: p.circleRateValue,
    transaction: tx._id,
    transactionValue: tx.transactionValue,
    paymentMethod: tx.paymentMethod
  }
```

Demo steps (Visualizer)

- Open **DataGraph** / **KnowledgeGraph**.
- Find an undervalued `RealProperty` , expand to its `RealEstateTransaction` .
- Explain the value mismatch and show the fields.

Test hooks

- Integration test asserts at least one undervalued transaction exists (see `tests/test_fraud_patterns_integration.py`).

Use Case 4: Benami / Proxy Identities ("Before vs After" Entity Resolution)

Story

The same real person appears as multiple records (typos, partial KYC, aliases). Entity Resolution collapses them into a `GoldenRecord`, revealing hidden connections and increasing risk visibility.

Graph pattern

- **Vertices**: `Person`, `GoldenRecord`
- **Edges**: `resolvedTo`
- **Optional supporting edges to demonstrate "hidden ties"**:
  - `residesAt`, `hasAccount`, `associatedWith`, `hasDigitalLocation`

Signals

- Multiple `Person` nodes share strong identifiers (`panNumber`) and/or correlated attributes.
- After resolution, a `GoldenRecord` has **≥ 2 inbound** `resolvedTo` edges.

Example AQL (starter)

```
WITH Person, GoldenRecord, resolvedTo
FOR g IN GoldenRecord
  LET n = LENGTH(
    FOR v, e IN 1..1 INBOUND g resolvedTo
      FILTER IS_SAME_COLLECTION("Person", v)
      RETURN 1
  )
  FILTER n >= 2
  LIMIT 10
  RETURN { goldenRecord: g._id, inboundResolvedTo: n }
```

Demo steps (Visualizer)

- "Before": show `dup_*` `Person` nodes with copied edges (messy).
- "After": follow `resolvedTo` edges to a `GoldenRecord` and show consolidated view.

Test hooks

- Integration tests assert:
  - `GoldenRecord` populated (>0)
  - at least one golden record has ≥2 inbound `resolvedTo`
  - no `Person` has more than one `resolvedTo` edge (see `tests/test_entity_resolution_integration.py`).

Use Case 5: Risk Seeds + "Guilt by Association" (Risk Intelligence Preview)

Story

A seemingly "clean" customer is risky because they are connected to risky entities (watchlist hits, high-risk businesses, or tainted transaction flows).

Graph pattern

- **Vertices**: `WatchlistEntity`, `Person`, `Organization`, `BankAccount`
- **Edges**: `relatedTo`, `associatedWith`, `transferredTo`
- **Key fields**:
  - `riskScore`, `riskDirect`, `riskInferred`, `riskPath`, `riskReasons`

Signals

- Risk flows from known bad entities to neighbors (bounded, decayed).
- Explanations are captured in `riskReasons` for auditability.

Example AQL (starter: find neighbors of high-risk nodes)

```
WITH Person, Organization, WatchlistEntity, relatedTo, associatedWith
FOR start IN Person
  FILTER start.riskScore >= 80
  FOR v, e, p IN 1..2 ANY start relatedTo, associatedWith
    LIMIT 50
    RETURN { start: start._id, v: v._id, edge: e._id }
```

Demo steps (Visualizer)

- Start from a high-risk seed (e.g., `WatchlistEntity` or `Person` with high `riskScore`).
- Expand outward and narrate why connected entities should be investigated.

Test hooks

- Risk Intelligence algorithms are defined in PRDs, but not enforced by current automated tests (add once risk scripts land).

Appendix: Where use cases are implemented today

- **Generator injections**: `scripts/generate_data.py`
  - `transferredTo.scenario`: `"cycle"`, `"mule"`, `"background"`
  - undervalued property logic: `transactionValue <= circleRateValue`
- **Integration tests**:
  - `tests/test_fraud_patterns_integration.py`
  - `tests/test_entity_resolution_integration.py`